Bước 3: XGBoost Model — Tài liệu chi tiết

Mục lục

- 1. XGBoost là gì?
- 2. Cách hoạt động — Ví dụ Argentina vs France
- 3. Tại sao chọn XGBoost?
- 4. Input & Output
- 5. Hyperparameters
- 6. Quá trình Training
- 7. Đánh giá Model
- 8. Feature Importance — Model học được gì?
- 9. Class Weights — Xử lý Draw
- 10. Hàm Predict & Output cho Bước 4
- 11. Output Files

---

1. XGBoost là gì?

XGBoost (eXtreme Gradient Boosting) là thuật toán machine learning dựa trên **ensemble of decision trees** (tập hợp nhiều cây quyết định). Ý tưởng cốt lõi:

> Thay vì dùng 1 cây quyết định lớn, dùng **500 cây nhỏ**, mỗi cây **sửa lỗi** của các cây trước đó.

Mỗi cây mới **không** học lại từ đầu, mà tập trung vào những trận mà các cây trước **dự đoán sai**. Sau 500 lần sửa lỗi, kết quả cuối cùng là tổng hợp của tất cả các cây.

---

2. Cách hoạt động — Ví dụ Argentina vs France

Giả sử model đang dự đoán trận **Chung kết World Cup 2022: Argentina vs France**.

Input features:

```
elo_diff               = +112     (Argentina mạnh hơn)
rank_diff              = +1       (Argentina hạng 3, France hạng 4)
home_attack_strength   = 1.65     (Argentina tấn công mạnh)
away_defense_strength  = 0.65     (France phòng thủ tốt)
expected_goals_diff    = +0.68    (Argentina kỳ vọng ghi nhiều hơn)
home_form              = 0.87     (Argentina phong độ cao)
away_form              = 0.80     (France phong độ cao)
is_neutral             = 1        (Sân trung lập Qatar)
```

Cây #1 (nhìn elo_diff)

```
elo_diff > 50?
├── Yes → nghiêng Home Win (Argentina Elo cao hơn nhiều)
└── No  → chưa rõ
```

Dự đoán sơ bộ: `Home Win 60% | Draw 25% | Away Win 15%`

Sai lệch: thực tế là Draw → **lỗi** được truyền cho cây tiếp theo.

Cây #2 (nhìn is_neutral)

```
is_neutral = 1?
├── Yes → giảm Home Win (không có lợi thế sân nhà)
└── No  → giữ nguyên
```

Cập nhật: `Home Win 50% | Draw 30% | Away Win 20%`

Cây #3 (nhìn away_defense_strength)

```
away_defense < 0.8?
├── Yes → France phòng thủ tốt → tăng xác suất Draw/Away Win
└── No  → giữ nguyên
```

Cập nhật: `Home Win 42% | Draw 35% | Away Win 23%`

... tiếp tục qua 500 cây ...

Mỗi cây tiếp theo tinh chỉnh thêm một chút, dựa trên lỗi còn lại.

Kết quả cuối cùng (tổng hợp 500 cây)

```
P(Home Win) = 32.5%
P(Draw)     = 36.0%  ← cao nhất
P(Away Win) = 31.5%

→ Prediction: Draw ✓ (đúng! thực tế Argentina vs France hòa 3-3)
```

Tại sao không chỉ dùng 1 cây lớn?

1 cây lớn (depth=20)	500 cây nhỏ (depth=3 mỗi cây)
Dễ overfit — "thuộc lòng" training data	Generalize tốt hơn
Không ổn định — thay đổi 1 trận ảnh hưởng toàn bộ	Ổn định — mỗi cây chỉ đóng góp 1%
Không có cơ chế tự sửa lỗi	Mỗi cây sửa lỗi cây trước

---

3. Tại sao chọn XGBoost?

Đặc điểm	Giải thích	Áp dụng trong project
Gradient Boosting	Mỗi cây mới học từ **lỗi** của cây trước	Cây #50 sửa lỗi cây #1-49 đã mắc

Đặc điểm	Giải thích	Áp dụng trong project
multi:softprob	Output là **3 xác suất** (Win/Draw/Loss) có tổng = 100%	Cho Monte Carlo simulation (Bước 4)
Non-linear	`elo_diff = +112` không đơn giản = thắng	Model học: neutral + elo cao → Draw cao hơn
Feature interaction	Tự phát hiện kết hợp features	`elo_diff cao + away_defense tốt + neutral` → Draw
Xử lý missing	Tự xử lý giá trị thiếu	Một số đội ít có lịch sử
Feature importance	Cho biết feature nào quan trọng	Dashboard hiển thị cho người dùng

---

4. Input & Output

Input: 8 Features (theo đề bài)

#	Feature	Nhóm	Ví dụ (ARG vs FRA)
1	`elo_diff`	Sức mạnh	+112
2	`rank_diff`	Sức mạnh	+1
3	`home_attack_strength`	Poisson	1.65
4	`away_defense_strength`	Poisson	0.65
5	`expected_goals_diff`	Poisson	+0.68
6	`home_form`	Form	0.87
7	`away_form`	Form	0.80
8	`is_neutral`	Bối cảnh	1

Output: 2 dạng

1. Xác suất theo lớp (Class Probabilities) — quan trọng nhất:

```
[P(Away Win), P(Draw), P(Home Win)]
# Ví dụ: [0.315, 0.360, 0.325]
# Tổng = 1.0 (100%)
```

2. Nhãn dự đoán (Predicted Label) — chọn xác suất cao nhất:

```
# P(Draw) = 0.360 là cao nhất
# → predicted_label = 1 (Draw)
```

Target Variable

```
result = 2  →  Home Win (đội nhà thắng)
result = 1  →  Draw (hòa)
result = 0  →  Away Win (đội khách thắng)
```

---

5. Hyperparameters

Giá trị tối ưu (tìm bằng GridSearchCV)

Parameter	Giá trị	Ý nghĩa
`n_estimators`	**500**	Số cây quyết định. 500 cây, mỗi cây đóng góp 1%
`max_depth`	**3**	Độ sâu mỗi cây. 3 tầng = tối đa 8 lá (2^3). Đơn giản, tránh overfit
`learning_rate`	**0.01**	Tốc độ học. Mỗi cây chỉ điều chỉnh 1% → học chậm nhưng chính xác
`subsample`	**0.8**	Mỗi cây chỉ dùng 80% dữ liệu training (random). Tránh overfit
`colsample_bytree`	**0.8**	Mỗi cây chỉ dùng 80% features (random). Tránh overfit
`min_child_weight`	**1**	Số mẫu tối thiểu trong 1 lá. 1 = linh hoạt
`objective`	`multi:softprob`	Bài toán phân loại 3 lớp, output xác suất
`eval_metric`	`mlogloss`	Đánh giá bằng log loss (đo chất lượng xác suất)

Tại sao learning_rate = 0.01 + n_estimators = 500?

Tưởng tượng vẽ tranh:

- **learning_rate = 0.3 + 50 cây** = 50 nét bút đậm → thô, dễ vẽ sai
- **learning_rate = 0.01 + 500 cây** = 500 nét bút nhạt → mượt, chính xác hơn

Tổng "lượng học" tương đương (`0.01 × 500 = 5` vs `0.3 × 50 = 15`), nhưng cách học chậm cho kết quả ổn định hơn.

Tại sao max_depth = 3?

Mỗi cây sâu 3 tầng chỉ xét **3 điều kiện** liên tiếp:

```
Ví dụ 1 cây trong model:

elo_diff > 50?
├── Yes: rank_diff > 20?
│   ├── Yes: home_form > 0.7?
│   │   ├── Yes → Home Win (xác suất cao)
│   │   └── No  → Home Win (xác suất trung bình)
│   └── No:  is_neutral = 1?
│       ├── Yes → Draw
│       └── No  → Home Win (nhẹ)
└── No:  away_defense < 0.5?
    ├── Yes: expected_goals_diff > 0?
    │   ├── Yes → Draw
    │   └── No  → Away Win
```

```
    └── No:  away_form > 0.7?
        ├── Yes → Away Win
        └── No  → Draw
```

GridSearchCV đã thử depth = 3, 5, 7 và chọn **3** là tốt nhất cho bài toán này. Depth lớn hơn → cây phức tạp hơn → overfit.

---

6. Quá trình Training

Dữ liệu

```
Tổng: 20,148 trận (2005-2026)
├── Train: 17,901 trận (2005-01 → 2023-12)
└── Test:  2,247 trận  (2024-01 → 2026-01)
```

Chia theo **thời gian** (không random) vì đây là dữ liệu chuỗi thời gian — model cần học từ quá khứ để dự đoán tương lai.

GridSearchCV + TimeSeriesSplit

```
Bước 1: Chia training data thành 5 folds theo thời gian

  Fold 1: Train [2005────2008] → Validate [2009────2012]
  Fold 2: Train [2005─────────2012] → Validate [2013──2015]
  Fold 3: Train [2005──────────────2015] → Validate [2016─2018]
  Fold 4: Train [2005──────────────────────2018] → Validate [2019─2021]
  Fold 5: Train [2005──────────────────────────────2021] → Validate [2022─2023]
```

```
Bước 2: Thử 36 tổ hợp hyperparameters

  max_depth     ∈ [3, 5, 7]           → 3 giá trị
  learning_rate ∈ [0.01, 0.05, 0.1]   → 3 giá trị
  n_estimators  ∈ [200, 500]          → 2 giá trị
  min_child_weight ∈ [1, 3]           → 2 giá trị
  ──────────────────────────────────────────────
  Tổng: 3 × 3 × 2 × 2 = 36 tổ hợp
  Mỗi tổ hợp × 5 folds = 180 lần train
```

```
Bước 3: Chọn tổ hợp có log_loss thấp nhất trên validation
  → Best: depth=3, lr=0.01, n_est=500, min_child=1
  → Best CV log_loss: 0.9541
```

```
Bước 4: Train final model trên toàn bộ 17,901 trận với best params
  → Áp dụng class weights: {Away Win: 1.16, Draw: 1.44, Home Win: 0.69}
```

```
Bước 5: Đánh giá trên 2,247 trận test (2024-2026)
  → Model chưa từng thấy data này trong quá trình training
```

---

7. Đánh giá Model

Metrics tổng hợp

Metric	Train	Test	Ý nghĩa
Accuracy	55.5%	**56.6%**	% dự đoán đúng
F1 Macro	0.526	**0.522**	Trung bình F1 của 3 class (cân bằng)
Log Loss	0.923	**0.891**	Chất lượng xác suất (thấp hơn = tốt hơn)

Train ≈ Test → **không overfit** (tốt!).

Accuracy 56.6% — có tốt không?

So sánh với baselines:

Phương pháp	Accuracy
Luôn đoán Home Win (class phổ biến nhất)	47.1%
Đoán theo Elo (đội Elo cao hơn thắng)	~50%
XGBoost model	**56.6%**
Lý thuyết tối đa (bóng đá có random rất lớn)	~60-65%

XGBoost cải thiện **+9.5%** so với baseline. Trong bóng đá, accuracy > 55% đã được coi là tốt vì tính bất định rất cao (đội yếu vẫn có thể thắng đội mạnh).

Classification Report chi tiết

```
              precision    recall  f1-score   support

Away Win       0.584      0.622     0.602       659
Draw           0.273      0.272     0.273       529
Home Win       0.705      0.678     0.691      1059

accuracy                            0.566      2247
macro avg      0.521      0.524     0.522      2247
```

Confusion Matrix

```
                    Predicted:
                    Away     Draw     Home
Actual Away Win:    410      156       93
Actual Draw:        178      144      207
Actual Home Win:    114      227      718

Accuracy per class:
  Away Win: 62.2% (410/659)
  Draw:     27.2% (144/529)
  Home Win: 67.8% (718/1059)
```

Phân tích

- **Home Win** dễ đoán nhất (67.8%) — khi có lợi thế Elo + sân nhà, khá chắc chắn
- **Away Win** khá tốt (62.2%) — đội khách mạnh vượt trội vẫn thắng được
- **Draw** khó đoán nhất (27.2%) — bóng đá hòa rất ngẫu nhiên, model vẫn bắt được 27% là tốt

---

8. Feature Importance — Model học được gì?

```
elo_diff                44.0%  ████████████████████████████████████
rank_diff               20.2%  ═══════════════════
expected_goals_diff     10.8%  ═══════════
is_neutral               9.9%  ═══════
away_defense_strength    5.1%  ═══
home_attack_strength     3.9%  ═
home_form                3.2%  ═
away_form                2.8%  ═
```

Insights

1. "Đội nào mạnh hơn" quyết định ~64%

- `elo_diff` (44%) + `rank_diff` (20%) = 64%
- Sức mạnh dài hạn là yếu tố quan trọng nhất

2. "Ghi bàn kỳ vọng" đóng góp ~11%

- `expected_goals_diff` kết hợp tấn công + phòng thủ của cả 2 đội
- Bổ sung thông tin mà Elo/Rank không capture

3. "Đá ở đâu" quan trọng ~10%

- `is_neutral = 1` làm giảm đáng kể lợi thế đội nhà
- World Cup hầu hết sân trung lập → tăng tỷ lệ Draw

4. Phong độ ngắn hạn ít ảnh hưởng ~6%

- `home_form` + `away_form` chỉ chiếm 6%
- Sức mạnh dài hạn quan trọng hơn phong độ 5 trận gần nhất
- Lý do: phong độ dao động ngẫu nhiên, trong khi Elo phản ánh năng lực thực

---

9. Class Weights — Xử lý Draw

Vấn đề

Phân phối kết quả không cân bằng:

```
Home Win: 8,597 trận (48%)  ← nhiều nhất
Away Win: 5,156 trận (29%)
Draw:     4,148 trận (23%)  ← ít nhất, khó đoán nhất
```

Nếu không xử lý, model sẽ "lười" — luôn đoán Home Win để tối đa accuracy, bỏ qua Draw hoàn toàn.

Giải pháp: Sample Weights

Tăng trọng số cho class thiểu số trong quá trình training:

```
class_weight = total / (n_classes × count_per_class)

Away Win (0): 17901 / (3 × 5156) = 1.16  (tăng nhẹ)
Draw     (1): 17901 / (3 × 4148) = 1.44  (tăng nhiều nhất)
Home Win (2): 17901 / (3 × 8597) = 0.69  (giảm)
```

Khi model đoán sai 1 trận Draw, "phạt" nặng hơn 1.44× so với đoán sai Home Win (0.69×).

Kết quả so sánh

Metric	Không weights	Có weights	Thay đổi
Accuracy	60.4%	56.6%	-3.8%
F1 Macro	0.444	**0.522**	**+17.6%**
Draw Recall	0.0%	**27.2%**	**+27.2%**
Draw F1	0.000	**0.273**	**+0.273**

Trade-off: accuracy giảm nhẹ (-3.8%) nhưng F1 tăng mạnh (+17.6%) và model **bắt đầu nhận diện được Draw**. Đây là trade-off xứng đáng vì:

- Xác suất Draw rất quan trọng cho Monte Carlo (Bước 4)
- Nếu model không bao giờ đoán Draw → Monte Carlo sẽ sai hoàn toàn
- Accuracy 56.6% vẫn cao hơn baseline 47.1%

---

10. Hàm Predict & Output cho Bước 4

Hàm predict_match()

```
def predict_match(model, features_dict):
    """
    Input:  {'elo_diff': 112, 'rank_diff': 1, ...}
    Output: {
        'probabilities': {
            'Home Win': 0.325,
            'Draw': 0.360,
            'Away Win': 0.315
        },
        'predicted_label': 'Draw',
        'predicted_code': 1
    }
    """
```

Ma trận xác suất cho Bước 4 (Monte Carlo)

Khi chạy Monte Carlo simulation, Bước 4 sẽ:

1. Gọi `predict_match(Argentina, France)` → `[0.325, 0.360, 0.315]`
2. Tạo random number 0-1
3. Nếu < 0.325 → Home Win
4. Nếu < 0.325 + 0.360 = 0.685 → Draw
5. Nếu ≥ 0.685 → Away Win
6. Lặp lại 10,000 lần → thống kê xác suất vô địch

---

11. Output Files

File	Đường dẫn	Mô tả
Trained Model	`models/xgboost_model.joblib`	Model đã train, dùng `joblib.load()` để nạp
Metadata	`models/model_metadata.json`	Features, params, metrics, feature importance
Source Code	`src/step3_xgboost_model.py`	Toàn bộ pipeline: load → train → evaluate → save

Cách sử dụng model

```python
import joblib
import pandas as pd

# Load
model = joblib.load('models/xgboost_model.joblib')

# Predict
features = {
    'elo_diff': 112,
    'rank_diff': 1,
    'home_attack_strength': 1.65,
    'away_defense_strength': 0.65,
    'expected_goals_diff': 0.68,
    'home_form': 0.87,
    'away_form': 0.80,
    'is_neutral': 1,
}
X = pd.DataFrame([features])
proba = model.predict_proba(X)[0]
# proba = [P(Away Win), P(Draw), P(Home Win)]
```

Source code: `src/step3_xgboost_model.py` | *Model:* `models/xgboost_model.joblib`